FIAP

Domain Driven Design - Java

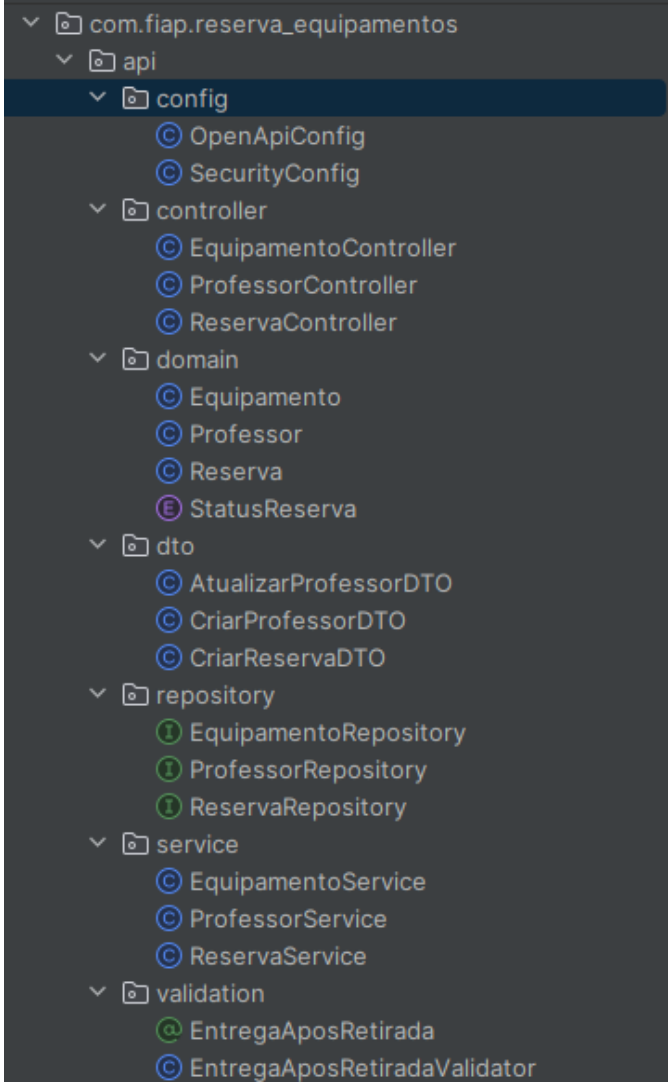
AGENDA

1. Spring Data JPA
2. Bean Validation (Jakarta Validation)
3. Spring Security

Iniciando nossa aula....

- Baixar do Teams o projeto - Reserva-equipamento.zip

Estruturando nosso projeto



Spring Data JPA

- Uma camada de abstração sobre o JPA que elimina grande parte do código de repositório (DAO). Você declara interfaces e o Spring gera as implementações em tempo de execução, incluindo consultas por convenção de nomes, paginação, ordenação, etc.

Spring Data JPA – Exemplo Prático - Domain

```java
package com.fiap.reserva_equipamentos.api.domain;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import lombok.Data;

@Data
@Entity
public class Equipamento {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Descrição é obrigatória")
    @Column(nullable = false)
    private String descricao;

    @Column(nullable = false)
    private Boolean ativo = true;
}
```

Spring Data JPA - Domain

- **jakarta.persistence.*:** anotações JPA para mapear a classe ↔ tabela.
- **jakarta.validation.constraints.NotBlank:** Bean Validation.
- **lombok.Data:** gera automaticamente getters/setters, equals/hashCode e toString.
- **@Entity:** diz ao JPA que esta classe é persistente. Sem @Table, o nome da tabela será o da classe.
- **@Id + @GeneratedValue(strategy = GenerationType.IDENTITY):** - Chave primária com auto-incremento do banco.

Spring Data JPA - Domain

- **@NotBlank(message = "Descrição é obrigatória") + @Column(nullable = false) -** Garante que descrição não seja vazia (validação de aplicação) e também não nula no banco (restrição de coluna).

Spring Data JPA – Exemplo Prático - Repository

```java
package com.fiap.reserva_equipamentos.api.repository;

import com.fiap.reserva_equipamentos.api.domain.Equipamento;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface EquipamentoRepository extends JpaRepository<Equipamento, Long> {
    List<Equipamento> findByAtivoTrue();
    List<Equipamento> findByDescricaoContainingIgnoreCase(String termo);
}
```

Spring Data JPA - Repository

- **Pacote de interface extends JpaRepository<Equipamento, Long>:** solicitar em tempo de execução que seja disponibilizado um repositório JPA para a entidade Equipamento cuja chave primária é Long”. Serão disponibilizados dezenas de métodos prontos: **save, findById, findAll, deleteById, paginação, etc.**

- **List<Equipamento> findByAtivoTrue():** Query derivada por convenção: busca todos onde o atributo ativo == true.

- **List<Equipamento> findByDescricaoContainingIgnoreCase(String termo):** query derivada: LIKE case-insensitive com %termo%

Spring Data JPA – Exemplo Prático - Service

```java
package com.fiap.reserva_equipamentos.api.service;

import com.fiap.reserva_equipamentos.api.domain.Equipamento;
import com.fiap.reserva_equipamentos.api.repository.EquipamentoRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class EquipamentoService {
    private final EquipamentoRepository repo;
    public EquipamentoService(EquipamentoRepository repo) { this.repo = repo; }

    public List<Equipamento> listarAtivos() { return repo.findByAtivoTrue(); }
    public List<Equipamento> buscar(String termo) { return repo.findByDescricaoContainingIgnoreCase(termo); }
}
```

Spring Data JPA - Service

- **@Service:** Marcar a classe como componente de serviço do Spring.
- **private final EquipamentoRepository repo; + construtor:** Injeção por construtor. O repo é imutável (final) e vem do Spring.
- **listarAtivos():** Retornar todos os equipamentos com ativo = true, chamando a query derivada do repositório findByAtivoTrue().
- **buscar(String termo):** Faz busca por descrição, case-insensitive, com “contém”: findByDescricaoContainingIgnoreCase(termo)

Bean Validation (Jakarta Validation)

- Padrão para validar beans (objetos) com anotações como @NotBlank, @Email, @Size, etc. O Spring integra isso nos controllers com @Valid e retorna erros automaticamente (400) quando inválido.
- Por que usar:
  - Centraliza regras de validação no modelo/DTO.
  - Mensagens de erro padronizadas e internacionalizáveis.
  - Menos código de “if/else” no controller.

Spring Data JPA – Exemplo Prático - DTO

```java
import jakarta.validation.constraints.FutureOrPresent;
import jakarta.validation.constraints.NotNull;
import lombok.Data;

import java.time.LocalDateTime;

@EntregaAposRetirada
@Data
public class CriarReservaDTO {

    @NotNull(message = "Equipamento é obrigatório")
    private Long idEquipamento;

    @NotNull(message = "Professor é obrigatório")
    private Long idProfessor;

    @NotNull(message = "Sala é obrigatório")
    private String sala;

    @NotNull @FutureOrPresent(message = "Retirada deve ser no presente ou futuro"
    private LocalDateTime horarioRetirada;

    @NotNull @Future(message = "Entrega deve ser no futuro")
    private LocalDateTime horarioEntrega;
}
```

Spring Data JPA - Validation

- **@Documented:** Faz a anotação aparecer no JavaDoc das classes que a usam.
- **@Constraint(validatedBy = EntregaAposRetiradaValidator.class):** Diz ao Bean Validation qual validador executa a regra quando a anotação é encontrada.
- **@Target({ ElementType.TYPE }):** A anotação só pode ser aplicada em tipos/classe (não em campo/método), pois a regra envolve dois campos do objeto.
- **@Retention(RetentionPolicy.RUNTIME):** A anotação está disponível em tempo de execução, para o mecanismo de validação lê-la.

Spring Data JPA - Validation

- **public @interface EntregaAposRetirada{...}**

- Declara a anotação em si, com:

  - **message() –** mensagem padrão quando a regra falhar;

  - **groups() –** permite grupos de validação;

  - **payload() –** metadados para consumidores.

Spring Data JPA – Exemplo Prático - Validation

```java
package com.fiap.reserva_equipamentos.api.validation;

import com.fiap.reserva_equipamentos.api.dto.CriarReservaDTO;
import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;

public class EntregaAposRetiradaValidator implements ConstraintValidator<EntregaAposRetirada, CriarReservaDTO> {
    @Override
    public boolean isValid(CriarReservaDTO dto, ConstraintValidatorContext ctx){
        if (dto.getHorarioRetirada() == null || dto.getHorarioEntrega() == null) return true;
        return dto.getHorarioEntrega().isAfter(dto.getHorarioRetirada());
    }
}
```

Spring Data JPA - Validation

- **implements ConstraintValidator<EntregaAposRetirada, CriarReservaDTO:** Este validador trata a anotação @EntregaAposRetirada aplicada a objetos do tipo CriarReservaDTO.
  - **isValid(CriarReservaDTO dto, ConstraintValidatorContext ctx):**
    - Se retirada ou entrega forem null, retorna true (outras anotações — @NotNull, @Future, etc. — já tratarão null).
    - Caso ambos existam, retorna: dto.getHorarioEntrega().isAfter(dto.getHorarioRetirada()).
    - → somente entrega > retirada é válido; se for igual ou antes, a validação falha.

Spring Data JPA – Exemplo Prático - API

```java
package com.fiap.reserva_equipamentos.api.controller;

import com.fiap.reserva_equipamentos.api.domain.Equipamento;
import com.fiap.reserva_equipamentos.api.service.EquipamentoService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController
@RequestMapping("/api/equipamentos")
public class EquipamentoController {
    private final EquipamentoService service;
    public EquipamentoController(EquipamentoService service) { this.service = service; }

    @GetMapping("/ativos")
    public List<Equipamento> ativos() { return service.listarAtivos(); }

    @GetMapping
    public List<Equipamento> buscar(@RequestParam(required = false, defaultValue = "") String q) {
        if (q.isBlank()) return service.listarAtivos();
        return service.buscar(q);
    }
}
```

Spring Security

- Framework de segurança do Spring para autenticação, autorização, filtros, segurança web, CSRF, etc.
- Por que usar:
  - Controle de acesso por URL, método, role.
  - Integra fácil com JWT/OAuth2, ou em memória para protótipos.
  - Boas práticas (senhas com BCryptPasswordEncoder, cabeçalhos, etc.).

Spring Secutiry– Exemplo Prático - Config

```java
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
                .csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                    // Swagger livre
                    .requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll()

                    // Regras específicas para Professores
                    .requestMatchers(HttpMethod.GET, "/api/professores/**").hasAnyRole("PROF", "ADMIN")
                    .requestMatchers(HttpMethod.POST, "/api/professores/**").hasRole("ADMIN")
                    .requestMatchers(HttpMethod.PUT, "/api/professores/**").hasRole("ADMIN")
                    .requestMatchers(HttpMethod.DELETE, "/api/professores/**").hasRole("ADMIN")

                    // Demais recursos já existentes
                    .requestMatchers("/api/equipamentos/**", "/api/reservas/**").hasAnyRole("PROF", "ADMIN")

                    // Qualquer outra rota exige autenticação
                    .anyRequest().authenticated()
                )
                .httpBasic(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    UserDetailsService users(PasswordEncoder encoder) {
        UserDetails prof = User.withUsername("professor").password(encoder.encode("123456")).roles("PROF").build()
        UserDetails admin = User.withUsername("admin").password(encoder.encode("admin123")).roles("ADMIN").build()
        return new InMemoryUserDetailsManager(prof, admin);
    }

    @Bean
    PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

Spring Security - Config

- **@Configuration + @EnableMethodSecurity:** Registra a classe como configuração e habilita anotações como @PreAuthorize nos services/controllers (ex.: cancelar() só ADMIN).

- SecurityFilterChain filterChain(HttpSecurity http). Define as regras de HTTP:
  - .csrf(csrf -> csrf.disable()) → desliga CSRF (ok para APIs stateless).
  - .authorizeHttpRequests(auth -> auth …) → mapeia quem acessa o quê:
  - permitAll() para Swagger: /v3/api-docs/**, /swagger-ui/**, /swagger-ui.html.

Spring Security - Config

- **Para /api/professores/**:**
  - GET: hasAnyRole("PROF","ADMIN")
  - POST/PUT/DELETE: hasRole("ADMIN")

Para /api/equipamentos/** e /api/reservas/**: hasAnyRole("PROF","ADMIN")

anyRequest().authenticated() → todo o resto precisa estar autenticado.

.httpBasic(Customizer.withDefaults()) → usa Basic Auth.

Spring Security - Config

- **UserDetailsService users(...)**
- Cria usuários em memória:
- professor / 123456 com role PROF
- admin / admin123 com role ADMIN
- (o hasRole("ADMIN") vira ROLE_ADMIN em background).
- PasswordEncoder passwordEncoder()
- Usa BCrypt (hash seguro). O Spring compara o hash ao autenticar.

Spring Secutiry– Exemplo Prático - Service

```java
package com.fiap.reserva_equipamentos.api.service;

import com.fiap.reserva_equipamentos.api.domain.Equipamento;
import com.fiap.reserva_equipamentos.api.domain.Professor;
import com.fiap.reserva_equipamentos.api.domain.Reserva;
import com.fiap.reserva_equipamentos.api.dto.CriarReservaDTO;
import com.fiap.reserva_equipamentos.api.repository.EquipamentoRepository;
import com.fiap.reserva_equipamentos.api.repository.ProfessorRepository;
import com.fiap.reserva_equipamentos.api.repository.ReservaRepository;
import jakarta.transaction.Transactional;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ReservaService {
    private final ReservaRepository reservaRepo;
    private final EquipamentoRepository equipamentoRepo;
    private final ProfessorRepository professorRepo;

    public ReservaService(ReservaRepository reservaRepo, EquipamentoRepository equipamentoRepo, ProfessorRepository professorRepo) {
        this.reservaRepo = reservaRepo;
        this.equipamentoRepo = equipamentoRepo;
        this.professorRepo = professorRepo;
    }

```

Spring Secutiry– Exemplo Prático - Service

```
27      @Transactional
28      public Reserva criar(CriarReservaDTO dto) {
29          Equipamento eq = equipamentoRepo.findById(dto.getIdEquipamento())
30                  .orElseThrow(() -> new IllegalArgumentException("Equipamento não encontrado"));
31          if (Boolean.FALSE.equals(eq.getAtivo())) throw new IllegalStateException("Equipamento inativo");
32
33          Professor prof = professorRepo.findById(dto.getIdProfessor())
34                  .orElseThrow(() -> new IllegalArgumentException("Professor não encontrado"));
35
36          // Verificar conflito de horário
37          boolean colide = !reservaRepo.overlaps(eq.getId(), dto.getHorarioRetirada(), dto.getHorarioEntrega()).isEmpty();
38          if (colide) throw new IllegalStateException("Já existe reserva para este equipamento nesse intervalo");
39
40          Reserva r = new Reserva();
41          r.setEquipamento(eq);
42          r.setProfessor(prof);
43          r.setSala(dto.getSala().trim());
44          r.setHorarioRetirada(dto.getHorarioRetirada());
45          r.setHorarioEntrega(dto.getHorarioEntrega());
46          r.setAtivo(true);
47
48          return reservaRepo.save(r);
49      }
50
51      @PreAuthorize("hasRole('ADMIN')")
52      @Transactional
53      public void cancelar(Long id) {
54          reservaRepo.findById(id).ifPresent(r -> { r.setAtivo(false); reservaRepo.save(r); });
55      }
56
57      public List<Reserva> listar() { return reservaRepo.findAll(); }
58  }
```

Application.Properties

```
spring.application.name=reserva-equipamentos
server.port=8080
spring.profiles.active=dev

# Swagger UI
springdoc.swagger-ui.path=/swagger-ui
springdoc.swagger-ui.display-request-duration=true
springdoc.swagger-ui.operationsSorter=alpha
springdoc.swagger-ui.tagsSorter=alpha
```

Application-dev.Properties

```
# H2 (dev)
spring.datasource.url=jdbc:h2:mem:reserva;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Application-prod.Properties

```
#ORACLE
spring.datasource.url=jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl
spring.datasource.username=RM
spring.datasource.password=ddmmaaaa
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
spring.jpa.hibernate.ddl-auto=none
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.OracleDialect
```

Desafio – Mão na Massa 💪

- Implemente as regras de CRUD completo (com banco de dados em memória – H2) para o item **“Equipamento”,** conforme tudo o que foi explicado na aula de hoje!

- Aos que optarem for fazer e entregar essa atividade **HOJE** ao depositar o exercício na tarefa – **API_Rest_Reserva_Equipamento** estarão automaticamente dispensados do CP2;

- Aos que não quiserem entregar realizarão o desenvolvimento em aula do CP2 no dia 27/04/2026, com tema que será liberado no dia!

OBRIGADO

FIAP

Copyright © 2026 | Professor Salatiel Luz Marinho | linkedin: linkedin.com/in/salatielmarinho
Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento, é expressamente proibido sem consentimento formal, por escrito, do professor/autor.

FIAP